CASIE Project: What We Did, Why, and How

Team Ultron · COE025 · Project 2 — Ranim Alyasein, Azaa Almousli, Tasnim Alibraheem, Saleh Almasaadi and Muhammet Çelik

We trained models that find the attacker, the victim, the tool and the affected data in cybersecurity news events. The best model reaches 0.620 F1 on the test set. This report explains every step, why we took it, and the questions we are likely to get in the presentation.

At a glance

Retrained on the corrected dataset (v2), RoBERTa gave the best result with 0.620 F1. Its difference from SecureBERT is not statistically significant. The CRF baseline stays at 0.525.

Class	Model	Precision	Recall	F1	95% CI	Latency (1 event)
Classical	CRF	0.639	0.446	0.525	0.499–0.553	12.3 ms (CPU)
General transformer	BERT (cased)	0.529	0.542	0.535	0.510–0.562	20.6 ms (T4)
General transformer	BERT (uncased)	0.540	0.603	0.570	0.546–0.593	21.4 ms (T4)
General transformer	RoBERTa	0.622	0.617	**0.620**	0.597–0.643	20.7 ms (T4)
Domain transformer	SecureBERT	0.598	0.631	**0.614**	0.592–0.636	19.6 ms (T4)

Four main findings:

1. **The task is not saturated.** Our best score is 0.62; the best result in the literature is 0.69. This is the opposite of the “saturated benchmark” situation the instructor warned about.
2. **Cybersecurity pre-training does not raise F1** (SecureBERT vs. RoBERTa, p = 0.51). SecureBERT is slightly less precise (−2.3 points, p = 0.018); the recall difference is not significant.
3. **Transformers find distant arguments.** For arguments 11–20 words from the trigger, recall is 21% for the CRF and 46% for SecureBERT.
4. **Surprise: uncased BERT beats cased BERT** (+3.4 F1, p = 0.002). The opposite of what we expected; discussed in the Results section.

1. The task: what we did and why we chose it

Our task is **event argument extraction (EAE)**: in a cybersecurity event, find the **role** of each participant.

Where we came from. The Project 2 framework was “make the computer label words” (token classification). Between NER and POS, the team chose NER, and the first plan was to find hacker groups, malware and tools in reports. The instructor then said by email that plain NER and POS are “baseline tasks” and would get reduced grades, so we kept the idea but made the task harder.

What is the difference? On the same sentence:

	Plain NER	**Our task (EAE)**
Sentence	“APT28 stole customer records from Equifax”	same
Output	APT28 → Organisation, Equifax → Organisation	APT28 → **Attacker**, Equifax → **Victim**, customer records → **Compromised-Data**
Question asked	“What is this?”	“What role does this play in the event?”
Number of classes	usually 3–4	**26 roles**, 5 event types

Why is it harder? The same company can be the victim in one event and the attacker in another. If a document contains several events, the model has to know which event it is looking at. So we give the model **one event at a time**: the event type (e.g. Databreach) and the word that signals the event (the **trigger**, e.g. “stole”).

Technically it is still token classification: every word gets a **BIO** label. B-Victim = first word of the victim, I-Victim = its continuation, O = not an argument. So we stayed within the project framework and increased the difficulty.

One sentence you can use in the presentation: *“Plain NER would be a baseline task, so we extended it to event argument extraction: instead of what an entity is, we predict what role it plays in a specific incident.”*

2. Dataset: CASIE and cleaning

We use Tasnim’s cleaned version of **CASIE** (Cybersecurity Event Extraction Corpus): 999 documents, 8,482 events, 22,694 arguments, 26 roles. Source: github.com/Ebiquity/CASIE (Satyapanich, Ferraro and Finin, AAAI 2020).

Why CASIE? It fits our cybersecurity idea exactly, it comes with event and role annotations, and it does not need to be annotated by hand. The instructor said “don’t bring an already cleaned dataset; cleaning effort earns points”, which makes this an advantage, because it still needed real cleaning.

Cleaning done by Tasnim:

1. Fixed 1,095 character-offset errors (the annotation pointed to text that did not match).
2. Removed a document that was an exact copy of another (annotation/1554.json); 999 of 1,000 documents remain.

3. Put two very similar documents in the same split, so one cannot be in training and the other in test.
4. Annotation-aware tokenization: 23 words were split to keep annotation boundaries. In the end all 31,176 annotations aligned exactly with words.
5. Separate BIO labels were produced for each event. Where the same argument has two roles in two events, it was kept as multi-label.
6. 80/10/10 split at document level (seed 42); events of the same document never end up in different splits.

Saleh prepared and compared a second version independently. The two were very close; Tasnim's became the official data because it includes extra leakage prevention.

Dataset v2. Ranim noticed that step 4 used gold annotation boundaries during tokenization, which a real system would not have. Tasnim fixed it: v2 tokenizes from the text only. Splits, documents and triggers are unchanged; only 25 of 22,694 arguments were affected. All models were retrained on v2, and all numbers in this report are v2.

Split	Documents	Events	Arguments
Train	799	6,877	18,445
Validation	100	869	2,140
Test	100	736	2,109

Role imbalance: the most frequent role is Victim (3,476 arguments), the rarest is Damage-Amount (42), an 80× difference. Damage-Amount has only 4 examples in the test set, so scores for rare roles are not reliable.

Why a document-level split? If one event of a news article is in training and another in test, the model could memorise the article and get an inflated test score. This is called **data leakage**.

3. Evaluation: what we measured and why

The main metric is **Arg-C F1**: a prediction counts as correct only if the start word, the end word **and** the role all match the gold annotation of the same event.

Why not accuracy? Most words are "O" (not an argument). A model that predicts O for everything would reach about 95% word-level accuracy. The instructor said "a baseline getting 90% accuracy makes the task meaningless", and reporting accuracy would have made our task look easy. Span F1 counts only real arguments.

What do precision, recall and F1 mean?

- **Precision:** of everything the model called an argument, how much was correct.
- **Recall:** of the real arguments, how many the model found.
- **F1:** the harmonic mean of the two; if either is very low, F1 drops too.

Other metrics we report:

Metric	What it tells us
Arg-I F1	Is only the span correct, ignoring the role? The gap to Arg-C = points lost to role confusion.
Macro-F1	Plain average of the 26 role F1s; rare roles count as much as frequent ones.
F1 per role and per event type	Where the model is good and where it is bad.
95% confidence interval	The test set has only 100 documents; how much could the score vary?
Paired bootstrap test	Is the difference between two models real or luck?

Why not seqeval? seqeval is the standard library for NER evaluation, but it assumes one label per word. In our data the same argument can carry two roles at once, and labels depend on the event. So we wrote our own evaluation directly on spans (`casie_eval.py`). On single-label data the result is identical to seqeval's strict mode.

What is bootstrap? From the 100 test documents we draw 100 documents at random, allowing the same document to be drawn more than once, and recompute the score; we repeat this thousands of times. The middle 95% of the scores gives the confidence interval. In the **paired** version, both models are compared on the same documents each time. We sample documents, not events, because events of the same document are not independent of each other.

Error types (every error falls into exactly one category):

Error type	Meaning
Missed	A gold argument was not found at all
Spurious	An argument that does not exist was predicted
Boundary	Correct role, but the span is too long or too short
Role confusion	Correct span, wrong role (e.g. Attacker instead of Victim)
Boundary + role	Both the boundary and the role are wrong

Model selection was done on the validation set only; the test set was used once per model.

4. Models: why these, how they work

Five models, each chosen to answer one question:

Model	What it is	Question it answers
CRF	Feature-based classical model, no GPU needed	How far can we get without transformers? (lower bound)
`bert-base-cased`	General-purpose transformer (Wikipedia + books)	How does a standard transformer do?
`bert-base-uncased`	Same model, lowercased text	Does capitalisation matter? (instructor's note)
`roberta-base`	General-purpose, trained on more and more varied data	Reference point for SecureBERT
`ehsanaghaei/SecureBERT`	RoBERTa + extra pre-training on cybersecurity text	Does domain knowledge help? (research question)

Why both RoBERTa and SecureBERT? They share the same architecture and the same tokenizer family; the only difference is the pre-training data. So the difference between them can be attributed directly to "domain knowledge". Comparing BERT with SecureBERT would mix architecture and data differences.

How does the CRF work?

A CRF (Conditional Random Field) labels each word from hand-designed features and also takes into account whether neighbouring labels fit together (e.g. I-Victim must follow B-Victim). The features we used:

- **Word features:** the word itself, its first and last 3 letters, its shape (Xxxx, dddd), whether it is capitalised or contains digits. The same for the 2 words before and after.
- **Event features:** the event type, distance to the trigger, whether the word is to the left or right of the trigger, whether it is in the same sentence as the trigger.

The event features make the CRF an argument extractor rather than an NER model: the same word gets different features for different events. The CRF looks at a window of ±100 words around the trigger.

How was input given to the transformers?

All four transformers were trained with exactly the same code (`casie_transformer.py`) and the same settings; only the model name changes. This is what makes the comparison fair.

An event is given to the model as a text pair:

- **Part A:** event type + trigger word, e.g. `Databreach stole`
- **Part B:** the document text, with the trigger wrapped in special markers: `... APT28 <t> stole </t> customer records ...`

`<t>` and `</t>` tell the model “look at this event”. Documents can be longer than the 512-token limit, so the window starts at the trigger and grows left and right until the limit is reached; the trigger is always inside. We checked: **100%** of gold arguments fall inside the window, so no argument is lost to the length limit.

For each word, the model picks one of 53 labels (O + 26 roles × B/I). Transformers split words into sub-pieces (e.g. “ransomware” → “ransom” + “##ware”), so the label is read from the first piece of each word.

Training settings (the same for all): AdamW optimizer, learning rate 3e-5, weight decay 0.01, warm-up over the first 10% of steps, effective batch 16 (8 × 2 gradient accumulation), at most 10 epochs, early stopping (stop if validation F1 does not improve for 2 epochs in a row), fp16, seed 42.

5. Training process: what happened and which decisions we made

All models were trained on Google Colab with a free T4 GPU; one epoch took about 3.6 minutes.

In order:

1. **The evaluation code was written first,** so that model results could be measured as soon as they arrived. It was tested on synthetic data: a perfect prediction gives 1.0, and every injected error falls into the right category.
2. **The training code was written and tested locally.** HuggingFace was not reachable from that environment, so it was tested with tiny BERT and RoBERTa models. This found and fixed a bug: events with no predicted arguments were dropped when reading predictions.
3. **CRF trained** (v1): test F1 0.512.
4. **BERT trained for 5 epochs** (v1): test F1 0.518. But the best validation score was at the last epoch, which means the model was still learning.
5. **Decision: 10 epochs + early stopping.** For fairness, BERT was retrained as well. The three models were trained one after another in one Colab cell (about 1.5 hours in total).
6. **Retraining on dataset v2.** After Ranim’s observation and Tasnim’s fix, all five models were retrained on v2. For the instructor’s note, `bert-base-uncased` was added, and the latency of every model was measured.

Model (v2)	Epochs run	Best epoch	Best validation F1	Test F1
BERT (cased)	7	5	0.526	0.535
BERT (uncased)	10	8	0.556	0.570
RoBERTa	7	5	0.619	0.620
SecureBERT	10	8	0.618	0.614

Why does this table matter? Two models stopped early; the two that used all 10 epochs stayed within 0.01 F1 over their last three epochs, so all of them had converged. Validation and

test scores are very close (RoBERTa 0.619 vs. 0.620), so model selection did not overfit the validation set.

What changed between v1 and v2? Scores moved by at most 1.3 points (CRF 0.512 → 0.525, BERT 0.541 → 0.535, RoBERTa 0.613 → 0.620, SecureBERT 0.613 → 0.614), all within the confidence intervals. The ranking and the main results stayed the same. Two small v1 findings did not hold in v2: BERT's advantage over the CRF (no longer significant), and SecureBERT's ±5-point precision/recall shift (now −2.3 / +1.3). With single-seed training, this much variation is normal.

Notes for running the notebooks:

- The "UNEXPECTED / MISSING" and "617 > 512" warnings at the start of training are normal: we discard BERT's pre-training head and add our 53-label classifier; the length warning comes from a count made only to compute the window.
- `model.safetensors` files are 400–460 MB. They are not needed for analysis and should not go on GitHub (100 MB limit).

6. Results and significance tests

The RoBERTa family (RoBERTa and SecureBERT) significantly beats the CRF and both BERTs, and is tied within itself. The scores are in the "At a glance" table.

Comparison	F1 difference [95% interval]	p	Precision diff. (p)	Recall diff. (p)
BERT-cased − CRF	+0.010 [−0.019, +0.038]	0.49	−0.110 (<0.001)	+0.096 (<0.001)
BERT-uncased − BERT-cased	+0.034 [+0.013, +0.056]	0.002	+0.011 (0.45)	+0.061 (<0.001)
RoBERTa − BERT-cased	+0.084 [+0.061, +0.108]	< 0.001	+0.093 (<0.001)	+0.075 (<0.001)
RoBERTa − BERT-uncased	+0.050 [+0.032, +0.067]	< 0.001	+0.082 (<0.001)	+0.014 (0.18)
SecureBERT − RoBERTa	−0.006 [−0.023, +0.011]	0.51	−0.023 (0.018)	+0.013 (0.21)
RoBERTa − CRF	+0.094 [+0.067, +0.121]	< 0.001	−0.017 (0.30)	+0.172 (<0.001)

How to read the table: if the interval does not contain 0, the difference is real. The p-value is "the probability of seeing a difference this large if the models were actually equal"; below 0.05 is accepted as significant.

What does it mean?

1. **RoBERTa and SecureBERT are equal** (p = 0.51). Domain pre-training did not raise F1; SecureBERT is only slightly less precise.

2. **The choice of base model matters more than domain adaptation.** With the same settings, RoBERTa beats BERT by 5–8 points.
3. **Cased BERT does not beat the CRF** ($p = 0.49$). It finds many more arguments (+9.6 recall) but loses as much in precision.
4. **RoBERTa finds 17 points more recall than the CRF at the same precision** (precision difference $p = 0.30$). The gain comes entirely from finding more arguments.

Cased or uncased? (instructor's note)

We expected cased models to win: attacker, victim and company names are capitalised, and CVE numbers have a fixed format. **The experiment showed the opposite:** uncased BERT beat cased BERT by 3.4 points ($p = 0.002$), almost entirely through recall. The biggest differences: Vulnerable_System_Version 0.29 → 0.52, Number-of-Data 0.32 → 0.60, Supported_Platform 0.00 → 0.57.

Still, we cannot conclude that "capitalisation is useless": the two best models (RoBERTa, SecureBERT) are cased. A likely explanation: part of cased BERT's vocabulary goes to upper/lowercase variants, so rare words are split into more pieces and recall drops. Since this is a single-seed result, we do not generalise from it.

Comparison with the literature (instructor's note)

The closest published work is **TextEE** (Huang et al., 2024): argument extraction on CASIE with gold triggers, exactly what we do.

Model	**Arg-C F1**
TagPrime-CR (best in the literature)	69.2
PAIE	65.7
X-Gear	63.4
Our RoBERTa	**62.0**
Our SecureBERT	**61.4**
BART-Gen	60.0
DEGREE	59.0
Our BERT-uncased	**57.0**
DyGIE++	56.0
OneIE	55.3
Our BERT-cased	**53.5**
Our CRF	**52.5**

Our results sit among the published models. They are not directly comparable: TextEE uses different splits (701/149/149 documents, averaged over 5 splits) and its best models are larger (RoBERTa-large). But the task is clearly not saturated: the best result in the literature is 69.

Latency (instructor’s note)

All transformers process one event in about 20 ms on a T4 GPU (about 145 events per second); they have the same size, so the same speed. The CRF needs no GPU and is faster on CPU (12.3 ms), but 9 points less accurate. So RoBERTa’s advantage over the CRF requires a GPU; among transformers, accuracy comes for free. Even without a GPU, the speed is enough for a live demo.

F1 by event type:

Event type	CRF	BERT	BERT-unc.	RoBERTa	SecureBERT
PatchVulnerability	0.604	0.544	0.622	0.650	**0.653**
Databreach	0.507	0.559	0.586	**0.645**	0.632
DiscoverVulnerability	0.535	0.550	0.597	**0.626**	0.625
Ransom	0.504	0.550	0.534	0.616	**0.622**
Phishing	0.468	0.454	0.475	**0.540**	0.525

Phishing is the hardest event type for every model. The likely reason: in phishing, the impersonated party (Trusted-Entity) and the target (Victim) are often confused.

7. Error analysis: where and why the models fail

The CRF’s main problem is not finding arguments at all; the transformers’ main problem is span boundaries.

Distribution of error types (test set, v2)

Error type	CRF	BERT	BERT-unc.	RoBERTa	SecureBERT
Missed	918	530	383	419	**378**
Spurious	**280**	582	631	404	492
Boundary (mostly “too short”)	**175**	336	349	301	298
Role confusion	**43**	55	64	55	60
Boundary + role	33	45	41	**32**	43

Role confusion is rare in all models. The gap between Arg-I and Arg-C F1 is 2–3 points: when a model finds the span, it usually gets the role right. The real difficulty is finding the span and drawing its boundaries correctly.

Finding 1: recall drops with distance from the trigger

43% of gold arguments are more than 5 words away from the trigger.

Distance to trigger	Arguments	CRF	BERT	RoBERTa	SecureBERT
1–5 words	1,211	0.61	0.66	0.75	**0.76**
6–10	408	0.32	0.48	0.52	**0.55**
11–20	312	0.21	0.37	0.44	**0.46**
21–50	160	0.06	0.21	0.28	**0.30**

The CRF sees ±100 words, so the information is available, but its local features cannot model long dependencies. A transformer's **self-attention** connects every word directly with all other words, which is why it does much better on distant arguments. This is the clearest answer to "why transformers?" in the project.

Finding 2: transformers end spans too early

Most boundary errors are "too short": the model starts the argument at the right place but ends it too early ("credit" instead of "credit card data", "passport" instead of "passport images"). The likely reason: each word is classified independently and the model predicts too few I- (continuation) labels. The CRF makes this error less often because it enforces label order; the natural fix is **BERT-CRF**. Long spans are hard in general: recall is 0.56–0.73 for 2-word arguments and 0.16–0.22 for arguments of 9+ words.

Finding 3: roles with a fixed pattern are easy, meaning-based roles are hard

Role	Best F1	Why
CVE	0.82 (SecureBERT)	Fixed format like CVE-2018-1234
Releaser	0.81 (BERT-uncased)	Usually a company name (Microsoft, Adobe)
Price	0.75 (SecureBERT)	An amount of money
Attack-Pattern	0.33	Long, sentence-like phrases
Purpose	0.26	Inferred from meaning, vague boundaries
Issues-Addressed	0.24	Same
Damage-Amount	0.00 (all transformers)	Only 32 training examples; same format as Price

SecureBERT's clearest domain gain is on CVE: 0.82, compared with 0.69 for the CRF and 0.59 for cased BERT.

Finding 4: some confusions appear in all models

- **Attacker ↔ Victim:** both are usually organisation names next to the trigger (e.g. in "the InterContinental Hotel Group data breach" the victim was predicted as the attacker).

- **Discoverer ↔ Vulnerable_System_Owner:** the one who found the vulnerability and the owner of the vulnerable system get confused. The two-role annotations in the dataset are exactly this pair; the problem partly comes from ambiguity in the annotation itself.
- **Victim → Trusted-Entity, Trusted-Entity → Tool:** in phishing, separating who was impersonated from who was targeted is hard.
- **Vulnerable_System_Version ↔ Patch-Number:** in phrases like “upgrade to version 2.3.32”, a version number can be both the vulnerable and the patched version.

8. Limitations

The results are reliable, but stating the points below openly in the presentation shows technical maturity.

- **Single seed.** Each model was trained once (seed 42). With different seeds, F1 can move by a few points. Bootstrap measures uncertainty in the test set, not randomness in training. So the SecureBERT–RoBERTa tie should be read as “no difference found”, not “definitely equal”.
- **No hyperparameter search.** All transformers were trained with the same settings (lr 3e-5). This makes the comparison fair, but each model could probably score slightly higher with its own best settings.
- **Triggers are given.** Our models receive the event and the trigger as given (gold triggers). A real system first has to find the trigger. In the demo app we do this with a trigger lexicon extracted from the training set. The lexicon finds test-set triggers with 0.50 F1 (precision 0.47, recall 0.54; thresholds chosen on validation), so end-to-end performance in the app is below 0.61.
- **Rare roles.** Damage-Amount (4 in test), Number-of-Victim (8) and Supported_Platform (11) scores can change by more than 0.1 with a single prediction.
- **Strict matching.** Predicting “credit card” instead of “credit card data” counts as completely wrong. This is a deliberate and standard choice, but it penalises the model’s “almost correct” predictions. The number of boundary errors shows how often this happens.
- **Annotation ambiguity.** Some confusions such as Discoverer / Vulnerable_System_Owner come from ambiguity in the annotation, not the model. Separating these would need errors to be inspected by hand; we only looked at examples.
- **No direct comparison with the literature.** Published CASIE results use different splits and different cleaning, so they cannot be compared one-to-one.

9. Likely questions in the presentation, with answers

1. **“Why not just NER?”**
    - Plain NER is a baseline task. We predict the role an entity plays in a specific event, with 26 roles. The same company can be the victim in one event and the attacker in another.
2. **“Why don’t you report accuracy?”**

- Most tokens are “O”, so even a useless model gets about 95% token accuracy. We use strict span + role F1, which only counts real arguments.

3. **“Is the task saturated?”**
   - No. Our best model reaches 0.62 F1, the CRF baseline 0.53. The best published gold-trigger result on CASIE (TextEE benchmark) is 0.69, and end-to-end systems reach only 0.45–0.61.
4. **“Did SecureBERT help?”**
   - Not in F1: it ties with RoBERTa ($p = 0.51$). It is slightly less precise (-2.3 points, $p = 0.018$). Its clearest gain is on CVE identifiers (0.82 F1).
5. **“Is the difference between models real or luck?”**
   - Paired bootstrap over test documents: RoBERTa and SecureBERT beat the CRF and both BERTs significantly; RoBERTa vs. SecureBERT and cased BERT vs. CRF are not significant.
6. **“Why cased models?”**
   - We tested it: uncased BERT beat cased BERT by 3.4 F1 ($p = 0.002$), mostly through recall. But the two best models (RoBERTa, SecureBERT) are cased, so case is not useless; for BERT the cased vocabulary splits rare words into more pieces. Single seed, so we don’t generalise.
7. **“What about latency?”**
   - All transformers process one event in about 20 ms on a T4 GPU (~145 events/s). The CRF needs no GPU and takes 12 ms on CPU, but is 9 F1 points worse.
8. **“Why do transformers beat the CRF?”**
   - Mostly on distant arguments. At 11–20 tokens from the trigger, recall goes from 21% to 46%. Self-attention connects distant words; the CRF only sees local features.
9. **“Were arguments lost because of the 512-token limit?”**
   - No. We centre the window on the trigger and checked: 100% of gold arguments fall inside the window.
10. **“Did the models converge?”**
    - Yes. Up to 10 epochs with early stopping; validation and test F1 are almost identical, so no overfitting to validation.
11. **“What is the main remaining error?”**
    - Span boundaries, mostly spans cut too short (“credit” instead of “credit card data”). A CRF output layer on top of the transformer would be the natural next step.
12. **“How did you prevent data leakage?”**
    - Document-level split, an exact duplicate removed, a near-duplicate pair kept in the same split, and (v2) tokenization from text only, without gold boundaries.
13. **“Why is Phishing the hardest?”**
    - Phishing mixes the impersonated party (Trusted-Entity) with the target (Victim); these two roles are frequently confused.
14. **“Why didn’t you use seqeval?”**
    - Two arguments carry two roles at once and labels depend on the event; seqeval assumes one tag per token. Our scorer matches seqeval’s strict mode on single-label data.

10. Files and terms

What is in the package (`TeamUltron_Project2.zip`):

File / folder	Purpose
`README.md`	Full project documentation (for the repo)
`Team_Summary.pdf`	Short team summary: results, instructor feedback, demo
`evaluation/casie_eval.py`	Evaluation, error analysis, bootstrap tests
`evaluation/casie_crf.py`	CRF baseline
`evaluation/casie_transformer.py`	BERT / RoBERTa / SecureBERT training
`evaluation/*.ipynb`	Colab notebooks (CRF, transformer, evaluation, demo)
`evaluation/PREDICTION_FORMAT.md`	How other models' predictions should be saved
`results/<model>/`	Tables by role, event type, error type, confusion, and example errors
`predictions/`	Validation and test predictions of every model
`runs/<model>/training_config.json`	Settings and validation scores per epoch
`figures/`	4 charts for the slides
`demo/`	Demo app guide, link and screenshots

Terms:

Term	Meaning
Trigger	The word that signals the event (“stole”, “patched”)
Argument	A participant or piece of information in the event (attacker, victim, date)
Role	The argument’s function in the event (26 roles)
BIO	B = beginning, I = inside (continuation), O = outside; the word-labelling format
Span	The consecutive words that make up one argument
Arg-C / Arg-I	Span + role correct / span correct
Fine-tuning	Training an already-trained model again for our own task
Pre-training	The model’s first training on large text, where it learns language; for SecureBERT this was done with cybersecurity text
Epoch	One full pass over the training data
Early stopping	Stopping training once the validation score stops improving
Self-attention	The transformer mechanism that relates each word to all other words
Data leakage	Test information leaking into training and inflating the score
Bootstrap	Measuring how much the score varies by repeatedly resampling the test set

Sources:

- CASIE repository — Satyapanich, Ferraro and Finin (2020), AAAI
- SecureBERT paper — Aghaei et al. (2022)
- CySecBERT paper — an alternative cybersecurity model (not used)
- TextEE benchmark — Huang et al. (2024), gold-trigger CASIE results